Figure 4.11 The middleware pipeline for the example application in listing 4.3. The order in which you add the middleware to `WebApplication` defines the order of the middleware in the pipeline.

The ASP.NET Core web server passes the incoming request to the developer exception page middleware first. This exception-handler middleware ignores the request initially; its purpose is to catch any exceptions thrown by later middleware in the pipeline, as you'll see in section 4.3. It's important for this middleware to be placed early in the pipeline so that it can catch errors produced by later middleware.

The developer exception page middleware passes the request on to the static-file middleware. The static-file handler generates a response if the request corresponds to a file; otherwise, it passes the request on to the routing middleware. The routing middleware selects a minimal API endpoint based on the endpoints defined and the request URL, and the endpoint middleware executes the selected minimal API endpoint. If no endpoint can handle the requested URL, the automatic dummy middleware returns a 404 response.

In chapter 3, I mentioned that `WebApplication` adds the `RoutingMiddleware` to the start of the middleware pipeline automatically. So you may be wondering why I explicitly added it to the pipeline in listing 4.3 using `UseRouting()`.

The answer, again, is related to the order of the middleware. Adding an explicit call to `UseRouting()` tells `WebApplication` *not* to add the `RoutingMiddleware` automatically before the middleware defined in Program.cs. This allows us to "move" the `RoutingMiddleware` to be placed *after* the `StaticFileMiddleware`. Although this step isn't strictly necessary in this case, it's good practice. The `StaticFileMiddleware`